Quá trình hình thành đề tài theo hướng thực nghiệm

---

1. Bài toán và dữ liệu đầu vào

1.1 Định nghĩa bài toán

Đề tài giải quyết bài toán **phát hiện cháy tự động từ luồng video camera**, với hai yêu cầu đặt ra từ đầu:

1. **Phát hiện sớm** — nhận diện lửa và khói trong khung hình trước khi đám cháy lan rộng.
2. **Đáng tin cậy** — không kích báo động giả liên tục (ánh đèn xe, ánh nắng, nén video) vì báo giả nhiều khiến người vận hành mất niềm tin rồi tắt hệ thống.

Hai yêu cầu này mâu thuẫn nhau theo nghĩa cổ điển (sensitivity vs. specificity), nên không thể giải quyết bằng một mô hình đơn lẻ. Phương án được chọn là tách hai lớp xử lý: *mô hình nhận diện per-frame + bộ lọc thời gian tích lũy evidence*.

1.2 Dữ liệu đầu vào — D-Fire v1

Dữ liệu huấn luyện sử dụng **dataset D-Fire** (phiên bản `dfire_v1`), một bộ dữ liệu công khai chuyên biệt về phát hiện cháy và khói trong môi trường thực tế.

Thuộc tính	Giá trị
Số lớp	2: `fire`, `smoke`
Định dạng nhãn	YOLO (bounding box, normalized)
Độ đa dạng cảnh	Trong nhà, ngoài trời, đêm, ngày, góc xa, góc gần
Nguồn ảnh	Camera giám sát, drone, video đám cháy thực tế

Lý do chọn D-Fire thay vì tự thu thập:

- Đã được kiểm định và dùng trong nhiều nghiên cứu học thuật — kết quả có thể so sánh được.
- Bao phủ đủ điều kiện khó: khói mờ không có lửa, lửa nhỏ trong khung hình lớn, ngọn lửa bị che khuất một phần.
- Có sẵn split train/val chuẩn, tránh data leakage.

---

2. Vì sao chọn YOLOv8 làm kiến trúc phát hiện

2.1 Các lựa chọn được cân nhắc

Kiến trúc	Ưu điểm	Nhược điểm
Classifier CNN (ResNet, EfficientNet)	Đơn giản, dễ train	Không định vị được vị trí lửa; không phân biệt fire/smoke trong cùng frame
Two-stage detector (Faster R-CNN)	Accuracy cao	Tốc độ chậm (~5 fps trên CPU), không phù hợp camera thời gian thực
One-stage detector: YOLOv5	Cộng đồng lớn, nhiều tài liệu	API cũ hơn, ít được cập nhật
YOLOv8 (ultralytics)	**Real-time, API nhất quán, anchor-free, dễ fine-tune**	Cần tài nguyên vừa phải

Lý do chọn YOLOv8:

- Anchor-free head → không cần tune anchors thủ công khi domain lửa/khói có tỉ lệ bounding box đa dạng.
- `ultralytics` API chuẩn hóa toàn bộ pipeline: `YOLO("yolov8n.pt").train(...)` — giảm công việc boilerplate, tập trung vào thực nghiệm.
- Hỗ trợ chạy inference trên cả CPU lẫn GPU với FP16 tự động, phù hợp môi trường triển khai không đồng nhất.
- Cho phép trích xuất feature map trung gian (dùng cho Grad-CAM) qua forward hook mà không cần sửa source.

2.2 Lựa chọn scale: n (nano) hay s (small)?

YOLOv8 có nhiều scale: n, s, m, l, x — tăng dần tham số và độ chính xác. Thực nghiệm được thiết kế để so sánh **có kiểm soát** giữa hai lựa chọn nhỏ nhất phù hợp với điều kiện triển khai trên CPU:

- `YOLOv8n` — ~3.2M tham số, ưu tiên tốc độ.
- `YOLOv8s` — ~11.2M tham số, ưu tiên accuracy.

---

3. Pipeline huấn luyện

3.1 Cấu hình chung (giống nhau ở cả hai run)

3.1 Cấu hình chung (giống nhau ở cả hai run)

```
dataset   : data/datasets/dfire_v1/dataset.yaml
epochs    : 50
batch     : 16
imgsz     : 640 × 640
optimizer : SGD với cosine LR decay (mặc định ultralytics)
seed      : 42
```

Mọi hyperparameter khác giữ nguyên mặc định của ultralytics — điều này là cố ý: biến duy nhất thay đổi giữa hai run là **kiến trúc backbone** (`yolov8n.pt` vs `yolov8s.pt`), đảm bảo so sánh công bằng.

3.2 Quá trình learning rate

Cả hai run dùng cùng lịch LR: warm-up tuyến tính từ epoch 1–3 (lr ≈ 0.0016) rồi cosine decay về ~5×10^{-5} ở epoch 50. Không có giai đoạn plateau bất thường — loss giảm đều qua 50 epoch.

3.3 Scripts huấn luyện

Script	Model	Run name
`scripts/train_baseline.py`	`yolov8n.pt`	`dfire_v1_n_baseline`
`scripts/train_yolov8s.py`	`yolov8s.pt`	`dfire_v1_s_baseline`

---

4. Chỉ số đánh giá

Ultralytics báo cáo các chỉ số sau trên validation set sau mỗi epoch:

Chỉ số	Ý nghĩa
Precision (B)	Trong số box dự đoán là lửa/khói, bao nhiêu % đúng
Recall (B)	Trong số box lửa/khói thật, bao nhiêu % được phát hiện
mAP@50 (B)	Mean Average Precision tại IoU=0.50 — chỉ số tổng hợp chính
mAP@50-95 (B)	mAP trung bình từ IoU=0.50 đến 0.95, bước 0.05 — đánh giá chất lượng localisation
val/box_loss	Loss định vị bounding box trên val set
val/cls_loss	Loss phân loại lớp trên val set

Chỉ số chính để chọn mô hình cuối là **mAP@50** — tiêu chuẩn thực tế cho object detection.

cân bằng giữa precision và recall trên ngưỡng IoU rộng rãi phù hợp với bài toán phát hiện cháy (vị trí bounding box không cần cực kỳ chính xác, quan trọng là phát hiện được vùng cháy).

---

5. Kết quả thực nghiệm — So sánh hai phương án

5.1 Đường cong loss trong quá trình train

YOLOv8n (84.5 phút / 50 epoch):

Epoch	train/cls_loss	val/cls_loss	mAP@50
1	2.292	1.943	0.404
10	1.417	1.477	0.581
20	1.228	1.330	0.660
30	1.105	1.214	0.688
39	1.012	1.161	**0.710**
50	0.829	1.152	0.705

YOLOv8s (132.4 phút / 50 epoch):

Epoch	train/cls_loss	val/cls_loss	mAP@50
1	1.978	2.067	0.379
10	1.382	1.443	0.603
20	1.177	1.281	0.658
30	1.023	1.170	0.706
39	0.906	1.117	**0.712**
50	0.695	1.106	0.708

Nhận xét: cả hai model có `val/cls_loss` hội tụ từ epoch ~30 và không tăng trở lại — không overfitting đáng kể trong 50 epoch.

5.2 Bảng so sánh kết quả tốt nhất (best checkpoint)

Tiêu chí	YOLOv8n	YOLOv8s	Chênh lệch
mAP@50	0.7095 (ep.39)	**0.7123 (ep.39)**	+0.28 pp

mAP@50-95	0.3855 (ep.39)	**0.3923 (ep.38)**	+0.68 pp
Precision	0.7124 (ep.39)	**0.7279 (ep.38)**	+1.5 pp
Recall	**0.6488 (ep.39)**	0.6685 (ep.39)	+1.9 pp
Thời gian train	84.5 phút	132.4 phút	+56.7%
Tham số	~3.2M	~11.2M	×3.5

5.3 Tốc độ inference thực tế

Đo trên `scripts/run_headless_sim.py` — chạy trên ảnh val set, sau warmup pass:

Môi trường	YOLOv8n	YOLOv8s
CPU (không có GPU)	~95–120 ms/frame	~160–210 ms/frame
CUDA GPU (nếu có)	~12–18 ms/frame (FP16)	~20–28 ms/frame (FP16)

Cả hai đều đủ nhanh để xử lý luồng camera 4–10 fps trên CPU — ngưỡng thực tế của camera IP giá rẻ và hệ thống giám sát nhúng.

5.4 Phân tích lỗi định tính

Khi chạy `run_headless_sim.py` trên các scene khác nhau, cả hai model cùng mắc một nhóm lỗi:

Loại lỗi	YOLOv8n	YOLOv8s
False positive do ánh đèn 1-2 frame	Nhiều hơn	Ít hơn
Miss khói mờ, mật độ thấp	Nhiều hơn	Ít hơn
Nhầm lửa nhỏ xa camera	Tương đương	Tương đương

YOLOv8s cải thiện rõ nhất ở khói mờ — phù hợp với bài toán phát hiện sớm (khói thường xuất hiện trước lửa).

---

6. Kết luận — Vì sao YOLOv8s là lựa chọn phù hợp nhất

6.1 Về mô hình phát hiện

YOLOv8s được chọn làm model cuối (`dfire_v1_s_baseline/weights/best.pt`) dựa trên ba lý do:

1. **Accuracy tốt hơn có ý nghĩa thực tế**: +1.5 pp precision, +1.9 pp recall, +0.68 pp mAP@50-95 so với YOLOv8n — đặc biệt quan trọng với khói mờ giai đoạn sớm.
2. **Tốc độ inference vẫn chấp nhận được**: 160–210 ms/frame trên CPU tương đương 5–6 fps — đủ cho camera giám sát 4–8 fps. Overhead so với YOLOv8n (~50 ms) không ảnh hưởng đến latency thực tế của pipeline vì bottleneck là I/O camera.
3. **Tham số tăng nhưng bộ nhớ vẫn hợp lý**: ~11M params, model khoảng 22 MB — triển khai được trên máy thông thường không cần GPU chuyên dụng.

Chọn YOLOv8m hoặc lớn hơn không hợp lý vì: val loss đã hội tụ ở cả hai model nhỏ, dấu hiệu thêm tham số sẽ không cải thiện nhiều trên D-Fire v1 với cấu hình 50 epoch; đồng thời inference trên CPU sẽ vượt ngưỡng thực tế.

6.2 Về thiết kế pipeline tổng thể

Kết quả thực nghiệm xác nhận lý do phân tách hai tầng xử lý:

```
YOLOv8s per-frame  →  TemporalFilter (window=1s, min_det=3)  →  AlertState machine
```

- **Per-frame model** không thể độc lập tránh false positive: ánh đèn xe có thể đạt conf=0.6–0.7 trong 1–2 frame. Không có ngưỡng confidence nào có thể loại bỏ hoàn toàn mà không ảnh hưởng recall.
- **TemporalFilter** giải quyết vấn đề này bằng tích lũy evidence theo thời gian: lửa thật xuất hiện liên tục ≥3 frame/giây, false positive từ ánh đèn chỉ xuất hiện 1–2 frame rồi tắt. Ngưỡng `min_det=3, window=1.0s` được chọn theo quan sát thực tế này.
- **AlertState machine 4 trạng thái** xử lý thêm trường hợp biên: lửa bị che khuất tạm thời (COOLDOWN không tắt cảnh báo ngay), phát cảnh báo lại khi đám cháy kéo dài (cooldown=30s).

6.3 Bảng tổng kết lý do chọn từng thành phần

Thành phần	Lựa chọn	Lý do thực nghiệm
Kiến trúc detection	YOLOv8 (one-stage)	Đủ nhanh trên CPU; anchor-free phù hợp tỉ lệ bounding box lửa đa dạng
Scale backbone	YOLOv8s	+1.9 pp recall so với YOLOv8n, đặc biệt với khói mờ; inference vẫn ≤210 ms/frame CPU
Dataset	D-Fire v1	Đủ đa dạng cảnh; nhãn chuẩn; có thể so sánh với nghiên cứu khác
Lọc nhiễu	TemporalFilter sliding window	False positive từ model per-frame không thể loại bằng ngưỡng confidence đơn thuần
Xác nhận cháy	AlertState (4 trạng thái)	Tình huống thực tế không nhị phân: lửa bị che,

	thai)	chạy dài dang, cooldown
Explainability	EigenCAM (SPPF layer 9)	Trực quan hóa vùng mô hình chú ý; không cần backward pass → nhanh hơn GradCAM cổ điển
Triển khai	FastAPI + GPU/CPU auto-detect	Chạy được trên máy không có GPU; FP16 tự động khi có CUDA